

Resolving technical issues in parameter mapping equivalence and related problems

or: Ordering with simple type equivalence

Document number: P1624R0
Date: 2019-06-17
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup: Core
Revises: None
Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Introduction

As explained in P1375R1, there are technical issues surrounding the definition of equivalence, especially with regards to non-dependent constraint expressions and in the determination of atomic constraint identity. Non-dependent constraints raise questions regarding what, exactly, they constrain; therefore, the author wrote P1375R1 based on a principle that partial ordering based on constraints should require a clear relationship between the constraints applied upon the candidates. Early feedback on that paper was that considering types that are otherwise the same to not be equivalent in the context of atomic constraint identity would be surprising in C++. The technical problems that originally motivated the exploration into P1375 remains, and this paper attempts to approach the issues from an alternative viewpoint that accepts simple type equivalence. Some ideas are presented, and the author believes that the expertise of CWG would be helpful in triaging the issues, evaluating the strategy, and in determining what design guidance may be needed.

Summary of the issues

The issues basically are:

- Overload resolution ([over.match.best]) asks us to prefer a more constrained non-template function using rules that order declarations based on their associated constraints ([temp.constr.order]), but “associated constraints” are defined for templates ([temp.constr.decl]) and not for functions.
- [basic.link] (and other subclauses) do not take trailing *requires-clauses* into consideration.
- Declaration matching ([over.dcl]) is based upon whether trailing *requires-clauses* are equivalent; however, *equivalent*, with respect to expressions ([temp.over.link]), is defined only for expressions involving template parameters.
- The equivalence of parameter mappings ([temp.constr.atomic]) is determined by the [temp.over.link] rules for expressions as well. For types, this makes no sense; and for expressions, this would require distinguishing between expressions that are functionally equivalent but not equivalent.
- How template parameters from one template is to be matched against template parameters in another template when they appear in substituted parameter mappings is not clearly defined.

Ideas for resolving the issues

Define *associated constraints* for non-template functions

In [temp.constr.decl], add a new paragraph:

The *associated constraints* of a non-template function is the normal form of the *constraint-expression* introduced by the trailing *requires-clause*, if any; otherwise, the function has no associated constraints.

Substitution of actual template arguments into constraints beyond that necessary for determining satisfaction may be necessary to determine the ordering. For example:

```
template <typename T> constexpr bool B = true;
template <typename T> concept C = B<T>;

template <typename T>
struct A {
    void f(void) requires true || C<T>;
    int f(void) requires C<int>;
};

int f(A<int> &a) { return a.f(); }
```

Similarly, for declaration matching:

```
template <typename T> constexpr bool B = true;
template <typename T> concept C = B<T>;

template <typename T>
struct A { void f(void) requires C<T>; };

template <> void A<int>::f(void) requires C<int>;
```

Such substitutions may lead to errors. Since these substitutions are not being performed as part of determining viability of candidates for overload resolution, the SFINAE process does not apply.

Differentiate functions by their *requires-clauses* as well

Various appearances of “parameter-type-list” in [basic.link], [namespace.udecl], [dcl.link], and [over.load] require updates to take *require-clauses* into account.

Make the determination of expression equivalence treat concept definitions as opaque

We can extend the definitions of equivalent and functionally equivalent cover expressions subject to normalization in general (not just those involving template parameters). Then, we add a further condition that an expression that may be subject to constraint normalization is functionally equivalent only if each *qualified-concept-name* that may be expanded by normalization would be considered to name the same type if, instead of a concept, a class template was named.

Change parameter mapping equivalence/substitution

For non-dependent (after substitution) members of the parameter mapping, consider types by type identity, and expressions by type and value.

For dependent type members of the parameter mapping, treat each layer of substitution as applying an alias template upon the type. The types are considered the same if declaration matching would consider two template functions parameterized in the same manner as the templates from which the parameter maps originate to be redeclarations if everything about them were the same except that the *parameter-type-list* of one consisted of a class template specialization taking one mapping as a type template argument, and the other consisted of a specialization of the same class template that takes the other mapping as a type template argument.

For dependent non-type members of the parameter mapping, treat each layer of the substitution that does not plainly forward the argument as applying a variable template upon the expression. Apply the redeclaration-based definition for types, except that the class template instead takes a non-type template argument.

If a template member of the mapping was named as a member of a dependent class, the alias template treatment applies to class.

Only consider ordering near-identical candidates

To avoid cases like the following (notice that the correspondence between template parameters between the two candidates differ depending on whether the arguments are deduced or explicitly specified), further restrictions need to be applied.

```
template <typename T> constexpr bool B = true;
template <typename T> concept C = B<T>;

template <typename T, typename U> struct A;

template <typename V, typename T, typename U>
int f(V, A<T, U> *) requires C<T>;

template <typename V, typename T, typename U>
void f(V, A<U, T> *) requires C<T> && C<V> { }

void g(A<short, long> *ap) { return f(0, ap); }
```

Namely, candidates that are specializations of function templates should be ordered based on their constraints only when the templates have the same name (including for *conversion-function-ids*), parameter-type-list, and template parameter lists. This would be consistent with the requirement for non-template functions:

```
template <typename T> constexpr bool B = true;
template <typename T> concept C = B<T>;

template <typename T>
struct A {
    bool operator==(const A &) const & requires true || C<T>;
};

bool operator==(const A<int> &, const A<int> &) requires C<int> && true;

bool f(A<int> &a) {
    return a == a; // ambiguous; parameter-type-lists are not the same
}
```